An Agent Based AI Approach to Solving Pacman

Samuel Godwin

K1630575

1. Introduction

The agent built within this project is an intelligent agent. More specifically, a reflex agent that interacts with its environment using an API and a MDP solver.

A reflex agent will implement actions in response to what the agent senses in the environment. This environment is a world with full observability, in some cases with the presence of predatory ghosts and in some cases without. Pacman will have access to the locations of all normal pacman game elements: food, capsules, ghosts, walls and the corners of the space. Due to the presence of ghosts and other processes influencing what happens, the world is dynamic. The pacman's movements in this program are all non-deterministic, so follow a non-deterministic motion model.

This agent is not myopic, in other words pacman surveys the long-term situation rather than achieve short-sighted gratification. To do so, this agent is an MDP-solver and is stochastic in nature - pacman is able to assess his situation but not always move in the 'best' direction for his end goal of winning. Rewards are short term (myopic), where as utility is long term.

I chose this approach fascinated by the concept of a situation which is more realistic and closer to the real world. Calculating values for utility at this point was necessary, as the world was non-deterministic and this has been something I have been curious to try.

2. Description

Pacmap uses a map, a representation of the real world, which I developed in my previous coursework, having kept in mind that it could be used in future, making it extensible and reusable. For this coursework I just had to add a numerical value of each space, as covered in labs (add the element that allows a value to be associated with each space).

I developed an implemented of each of three methods for solving my MDP, with the intention on comparing them for my particular

2.1. Strategy

Efficient run speed -An important strategy was ensuring the run speed was a reasonable level whilst not forfeiting performance. This was achieved by a lower gamma (0.66), the key was to have a gamma that was not of too high a value (as discussed elsewhere in this report).

Rewards for each type of state/policy system - As ghosts are the most scary and least desirable element, of, course, Pacman will try to escape from ghosts if in danger. Initially I has chosen food as the most desirable element (as discussed elsewhere in this report), but in the final version of my program, capsule have the highest reward with food being the second highest. Pacman also has a reward value, to ensure that he is always inclined to move away from his current position. It seemed appropriate that empty squares with no food were similarly undesirable.

Policy iterations /modified policy iterations - These methods showed to contain the best move to take for every state across a layout, in a static world. As our world isn't static – ghosts move- we do the recalculation each time. The MDP is built around the idea that everything is static, and must take into account squares ghosts may move into. For this reason I take neighbouring states around ghosts into account when selecting my rewards. The strategy that I use in the final version of my program is the standard policy iteration, for reasons explained throughout this report (and in particular the results section).

Surrounded pacman - When Pacman has two enemies on either side, I have noticed that the policy may cause him to momentarily stop in his tracks. This being a behaviour I implemented manually in the previous coursework, I found this particularly interesting. I

have seen this behaviour lead to Pacman's survival and escape from situations in which he is surrounded by ghosts, numerous times.

2.2. Methodology

All information used was accessed entirely through the API.

I developed utility iteration, policy iteration and modified policy iteration as three separate methods (valueIteration, PolicyIteration and modifiedPolicyIteration respectively). Each of these approaches to the MDP problem adopt the same core structure and make use of the Bellman equation: a discount (gamma) and a number of lists and dictionaries (e.g. for actions North, South, West and East) are initialised.

Until convergence (or a 'stop-early' condition), I repeat the following process for every state: initialise a list of coordinates of states neighbouring the current state and set that neighbour equal to the current state, if the neighbour is not a state in the pathMap (i.e. is a wall). Then, for each possible action that could be taken from the current state, I do the calculation.
Which I break down in my code as first a calculation for the sum of an action's probability multiplied by the previous utility of the neighbour that action would move it to (for each of north, south, west and east).
The utility for the current iteration is then calculated as the maximum expected output from this calculation – i.e. the maximum expected value.

The below functions are as separate functions only for clarity of which is which, not for any reasons of pragmatism (as just one of the three methods could be written in the code). These functions can then be called in the getAction method whenever the agent is ready to make a move. getProbabilities is used to collate and return the probabilities that would be associated with the likelihood of a given intended action, since this is a non-deterministic world.

All mathematical diagrams/equations taken from lecture notes (lecture 4).

<u>Value Iteration (Utility Iteration)</u>

Value iteration will leave me with the value of a state under the optimal policy.

In the case of **valueIteration**, this maximum expected value is multiplied by discount and added to the reward for the current state in question and the process iterates through until utility == previousUtility, i.e. the values converge. A threshold value is used to end the

iteration early, when the difference between the utilities is small enough to consider them converged.

Process Followed in this Method

1. Set utilities to an arbitrary value U_0
2. Update utilities according to:

$$U_{i+1}(s) \leftarrow R(s) + \gamma \max_{a \in A(s)} \sum_{s'} P(s'|s,a) U_i(s')$$

   until the utilities no longer change

Policy Iteration

Given the utility of a state under the optimal policy, the optimal policy can be found.

In **policy iteration**, a similar process and maths is used, however in two parts, policy evaluation and policy improvement. In evaluation it should be noticed that the probabilities used concern current policy for the state, and not just an action. Rather than the maximum expected value, the discount is applied to the sum and reward for the state added, updating the utility, like in the previous function. In improvement the index of the maximum expected value is instead assigned to each state in the policy, thus updating it each iteration - until it no longer changes, or is optimal. Both for loops execute each iteration. This was the chosen method.

Process Followed in this Method

- Put these together to get:
- Starting from some initial policy π_0 we do:
  1. Policy evaluation
     Compute:

     $$U_i(s) = R(s) + \gamma \sum_{s'} P(s'|s, \pi_i(s)) U_i(s')$$

     for every state.
  2. Policy improvement
     Calculate a new policy π_{i+1} by applying:

     $$\pi_{i+1}(s) = \arg \max_{a \in A(s)} \sum_{s'} P(s'|s, a) U_i(s')$$

     for every state s.

  Until convergence.

Modified Policy Iteration

In the case of **modifiedPolicyIteration**, the only difference between that and policyIteration is that there is a fixed number of iterations for the evaluation part, allowing it to update further before going to the improvement step, although this makes it an approximation. Like with policyIteration, both for Loops execute each iteration until optimal.
Alterations are that for this, modifiedPolicyIteration uses a set number of iterations in its 'approximate policy evaluation' step. The set number of iterations here makes it an approximation, as it isn't left to loop infinitely or until convergence.
I tested and experimented with many set numbers of iterations for this part to discover that any number of iterations above 10 would just take too long to compute to be a viable option.

Process Followed in this Method

- Starting from some initial policy π_0 we do:
  1. Approximate policy evaluation
     Repeat

$$U_{i+1}(s) \leftarrow R(s) + \gamma \sum_{s'} P(s'|s, \pi_i(s)) U_i(s')$$

     a fixed number of times.
  2. Policy improvement

$$\pi_{i+1}(s) = \arg \max_{a \in A(s)} \sum_{s'} P(s'|s, a) U_i(s')$$

     for every state s.

  Until convergance
- Often more efficient than policy iteration or value iteration.

Functions Used Within the Main Function

rewardMap - The result of the function 'rewardMap' is assigned to a variable. This contains information I chose for state rewards having assigned each state in the layout with a numerical value for reward. A dictionary mapping states (coordinates) to their utilities is given an initial value of 0 for every state.

createMapOfAllPaths - As in my previous coursework, I also made use of a method called 'createMapOfAllPaths'. I calculated both width and height in this method via the corners accessible through the API.

Finally, I have decided to keep all three of these methods in the code for the moment for purposes of it being possibly assessed, but I will only used Policy Iteration because it had the best win rate.

The World Considered as MDP

Pacman's world is considered as an MDP (markov decision process).
MDPs and value iteration are methods for dealing with non-deterministic actions, but they only give optimal solutions in static worlds. The utilities that are computed by value iteration, and the policy that is determined by those utilities, are only optimal if the rewards don't change. With ghosts moving around, the rewards change dramatically every step.
Due to the computer intensive process of updating rewards and and running value iteration for all steps, the efficiency and performance have to be considered. I considered this problem by building three strategies, which I subjected to rigorous testing and experimentation to choose what I considered to be the most effective for this application.

Corrections to Code

For a long time during the coding process, my utility iteration function was incorrect without my noticing. The values would all converge to become the same, meaning that if left for long enough, every utility for every state would be identical.
I assumed this was some complex outcome of the maths and stopped the iteration before convergence started to occur, allowing me to see and use values, which seemed right (ghost utility much lower than food, for example). The only concern was the values did not propagate. Further inspection into the maths and rewriting the method caused it to run correctly, converging to values which were distinct depending on state and also propagating as expected.

It is worth noting that at this point, the rewards for states and the way they increased by appeal of a state prioritised food above all others states and thus food had the highest value. I considered food as more important than capsules because I wanted much more the game to be finished than to reach a high score eating ghosts.

In development of policy iteration, too many of my utilities were the same with not enough difference or variation. The policy including a large number of unreasonable actions (i.e. north into walls - I had drawn arrows on to diagrams reflecting policy outcome) with the vast majority being the direction 'north'. I did not want to tamper with the function by removing values of s' (from illegal moves) or setting them as very negative if not a valid state, as I figured this would affect every other state in a way I would not want. I

experimented in pursuit of solving this issue, making a dictionary of pseudo-utilities in the policy calculations (which were - importantly - all distinct different values).

How rewards changed things

In order to solve this I carried out more testing. The testing lead to making a dictionary of pseudo-utilities, which were, importantly, all distinct different values. When used in my function for policy, this lead to a policy that was much more spread out/much more varied directions over the whole collection of states. I realised that the reason my utilities were not varying enough was because the highest reward was *everywhere* (food). I made capsule the highest reward and food second highest and this solved the problem, giving a policy that was much more varied and reliable.

On observation of my layouts to test whether my policy for each state seemed reasonable, I noticed that at the capsules in top left, the policy move was an unreasonable North into a wall. Thankfully, this problem, from my observation, would solve itself. I realised the return was north because the north value is the value of the current state - which had the highest reward. When Pacman was actually at that state, he went East because of his lower reward value influencing the utility of the state. The high utility of the capsule therefore does not project Pacman into the wall.

Policy being wrong - The issue that was flagged by this problem and ultimately led to it being identified and solved, was the policy including a large number of unreasonable actions with the vast majority being the direction 'north'.

The Idea of Stopping Convergence Early

While developing utility iteration, I cut value iteration off early, aiding efficiency but making results more approximate. In testing, i noted that the values between different states (i.e. ghosts to food) were different enough and seemed satisfactory. I cut off the convergence early, enabling the iteration to finish more quickly and thus being more efficient.

Other Values

I set rewards to specific values according to their priority to the agent. the ghosts and their neighbours held a value of -100. This was the lowest value of the rewards, as the ghosts were to be avoided at all costs. The agent's current position held a value of -1, which was

slightly negative to encourage it not to remain in the same state. Empty states' rewards were -0.04, which was intended to be higher than pacman's position (that is moving to an empty space is more favourable than standing still), but still lower than the ghosts and their neighbours. The food was given a value of 10, as it is the most common state on the mediumClassic layout, and while being important for pacman to go to, a value much higher would problems with utility value propagation. the capsule was given the highest reward (250) to act as a 'goal' for the agent to move to, while still collecting the food.

I saw that pacman lost time when using a high gamma. Despite a low gamma of .05 (see Results section) performing extremely well on smallGrid, I noticed this did not perform anywhere near as well on mediumClassic.

Other idea I contemplated

'Experiment with different decision criteria for selecting actions. Do you see any difference in performance?' do you think this is literally stuff like, if pacman is too proximity to ghosts just ignore the policy and stay the other side of the map, perhaps until ghosts are edible, and then follow the policy (perhaps with edible rewards) so the aim for pacman would be avoid ghosts/collect food but aim for capsules until ghosts are edible and then get to the other points of the map

A Note on the Lacking of Resetting Values Between Games

With regard to resetting values to empty between consecutive runs - so that Pacman will fairly approach each time without relying on information gained in the previous run, like in the previous coursework - I did not deem this necessary. This is because Pacman builds no portfolio of advantage-giving info while he plays. This is given the MDP nature of the problem.

The correct implementation of the policy how I've built it is also inferred through Pacman's behaviour whenever a ghost is near; Pacman flees in whichever opposite direction is available, with the main issue being the non-deterministic world and a possible 0.1 chance every now and then of Pacman stopping in his tracks (leading to his demise).

A cool sentence: This would not greatly enhance performance on the layout mediumClassic, which already has a high success rate, so extra time was not given to developing this feature.

The Process of Choosing a Value of Gamma

I observed that it would take some experimenting of the gamma value to get an outcome that makes Pacman perform well, this possibly would this depend on size of layout as well.

I observed that Pacman got stuck anywhere with only one food left. To solve this, I tried to multiple things that didn't work: including
-not capsule neighbours,
-not food neighbours,
-not 'lots of empties in one area' somehow being calculated as lower, like the bigger the mass of negatives the lower they have somehow, like if they have full empty neighbours or something.

Finally, I chose to test to find the best value of gamma This worked well on smallGrid but not on medium classic, so I experimented with them further, until I found one that was the most balanced that I could observe on both layouts (data/results/table for this)

Realised lower gamma values were doing horribly, 0/10 a lot, like 1-7/50. So figured I'd go the other way, tried 0.9, and got one decent run, but took a long time to complete.

So tried 0.75 which still took quite some time and I finally remembered 0.5 was the starting point, so opted for 0.6. In the end 0.66 seemed the most efficient.

I tried 0.75 with the smallGrid and it was 13/50, which was about 20% success rate and that also helped. However, I wanted to get lower for good smallGrid and higher for good mediumClass so 0.66 seemed the best option however, I did not compare this with smallGrid. These 0.66 gamma, the reward values you'll see used, and normal policy, got 3/10 or 4/10 three times, WITHIN 5 mins (around 3 to be more precise).

3. Results

3.1. Getting the results

I have used a series of tests to assess my code for functionality under two scenarios. The two scenarios are mediumClassic and smalMDPlGrid.

MediumClassic is a three segment grid full of food, 2 capsules and an empty small box, and two ghosts. This map was useful for seeing whether my MDP would allow pacman to:
a)evade ghosts
b)locate the food and win the game.

SmallGrid is very small one segment grid with two pieces of food in and one ghost. This map was useful for seeing whether my MDP would allow pacman to:
a)evade ghost
b)locate the food when food is more sparse and win the game.

3.2. Reporting and analysing the results

For the results, please refer to the below table. All tests were ran 10 times iteratively using **-n 10**.

Table 1. A Table Showing the Success Rates of the Agent in Specified Grids

Name	**Success Rate (%) 1**	**Success Rate (%) 2**	**Success Rate (%) 3**	**Average Success Rate (%)**
mediumClassic	20	30	40	30
smallGrid	60	40	40	46

The conditions that these maps were ran with is as follows:
gamma 0.66, ghost -100, ghostneighbours -100, food 10, pacman -1, capsule 250, empty -0.04.

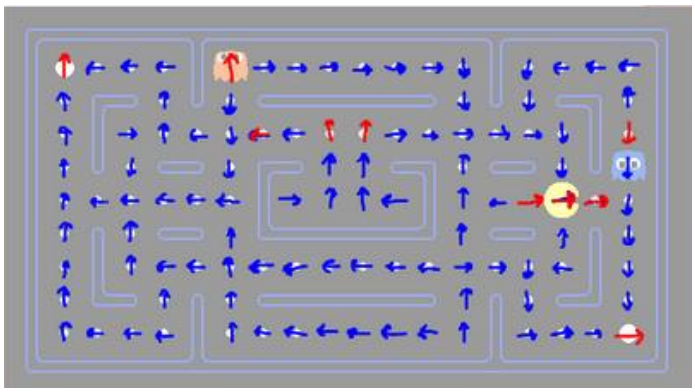
On analysis of the results, the MDP solver for pacman in the mediumClassic grid shows that pacman is capable of evading the ghost, consuming food and winning with an average 30% win rate. This ensure the value iteration method does allow pacman to navigate the grid despite his initial surroundings having the same elements. This confirms the long-term goal technique I hoped to achieve through the use of MDP.

On analysis of the results, the MDP solver for pacman in the SmallGrid, shows that pacman is capable of evading ghosts in a smaller map, and locating the food when food is more sparse and winning the game with an average 46% win rate. . This map highlights pacman's capability of using a long-term, value difference approach to finding food. There would be more of a dramatic intitial difference between each square in this map in comparison to the mediumClassic grid (as it had food all throughout). From my own observation of this grid, I saw that my policy iteration won games in smallGrid quite efficiently.

Finally, to test my method's function, I printed to console the final set of utilities for each move. The aim was to monitor whether they appear correctly, because of how they have shown to vary to each other, and the outcome of desirability I would want to what is in each state (e.g a ghost having lower utility than food). With this, I was able to confirm my method was functioning correctly.

The performance across runs of the exact same configuration can vary quite considerably (as seen in table 1) when compared to the previous coursework, and so win rates are quite erratic. I can only follow consistency to determine results as reliable evidence in this case. The below diagram helps to visualise the result we saw printed.

<u>A Diagram Showing the result of Policy iteration working as Expected</u>



One thing to note was the success of modified policy iteration which was more functional with smallGrid but less functional with mediumClassic. It also, did not perform well on iterations. This was possibly due to the number of iterations in modified policy iterations, because that determines how approximate the result is. I worked on getting the method to work at the most reasonable speed, on both maps, by value of gamma. For example, 20 iterations modified policy iterations runs very slow on mediumClassic, with almost 5 minutes to complete 1 game. If i were to improve this, I would lower the gamma as well as

the iterations, although lowering the iterations (as tested) did not make marginal difference. As the decrease in performance of lowering the gamma value would not be worth it, I decided that spending additional time on this issue would not be the most efficient option.
Therefore, the policy iteration method was chosen as the final option to ensure completions of this courseworks requirements.